



KICKOFF SPARK

Security Review



MARCH, 2026

www.chaindefenders.xyz
<https://x.com/DefendersAudits>

Lead Auditors



PeterSR



0x539.eth

Table of Contents

1. Protocol Summary
2. Disclaimer
3. Risk Classification
4. Audit Details
 - Commit Hash
 - Scope
 - Roles
5. Executive Summary
 - Issues Found
 - Issues Statuses
 - Security Grade
6. Findings
 - High
 - Medium
 - Low
 - Informational
 - Gas

Protocol Summary

Permissionless token launch product with bonding curve price discovery and automatic migration to Aerodrome V2 liquidity on Base.

The flow is the following:

1. Token Creation — Anyone calls `factory.createToken(name, symbol)`. Factory deploys token + bonding curve, transfers entire supply to the curve.
2. Bonding Curve Phase — Users buy/sell via `bc.buy{value: ethAmount}(minTokensOut, deadline)` and `bc.sell(tokenAmount, minEthOut, deadline)`. Fees: 4% sparks (reflection) + 1% creator (WETH) + 1% foundation (WETH).
3. Migration — When market cap reaches the migration threshold (default 5 ETH), the buying transaction automatically:
 - Burns excess tokens
 - Creates an Aerodrome V2 TOKEN/WETH pool
 - Locks LP tokens permanently in SparkLPLocker
4. Post-Migration — Token trades on Aerodrome V2. Same fee structure applies. 0.3% LP trading fees go to the protocol admin.

Disclaimer

The Chain Defenders team makes all effort to find as many vulnerabilities in the code in the given time period, but holds no responsibilities for the findings provided in this document. A security audit by the team is not an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the implementation of the contracts.

Risk Classification

Impact/Likelihood	High	Medium	Low
High	High	High	Medium
Medium	High	Medium	Low
Low	Medium	Low	Low

Audit Details

Commit Hash

b7e57c97442d1986f66369d2b24a0829970ce0ae

Scope

Id	Files in scope
1	src/KickoffFoundation.sol
2	src/SparkBondingCurve.sol
3	src/SparkFactory.sol
4	src/SparkLPLocker.sol
5	src/SparkToken.sol
6	src/SparkTokenDeployer.sol

Roles

Id	Roles
1	Admin
2	User

Executive Summary

Issues Found

Severity	Count	Description
High	1	Critical vulnerabilities
Medium	1	Significant risks
Low	5	Minor issues with low impact
Informational	2	Best practices or suggestions
Gas	1	Optimization opportunities

Issues Statuses

Id	Title	Status
H-01	Auto-swap Reverts On Paused Aerodrome Pool Bricking All Token Transfers	Fixed
M-01	Ghost ‘_rOwned’ Balance Drift On Excluded Addresses	Acknowledged
L-01	Plain Text Private Key In Deployment Scripts	Acknowledged
L-02	‘setMcapParams’ Allows Near-zero Migration Gap Enabling Instant Migration	Acknowledged
L-03	‘swapThreshold’ Fixed At Deployment And Becomes Misaligned With Circulating Supply	Acknowledged
L-04	‘confirmDeposit’ Lacks ‘nonReentrant’ Protection	Fixed
L-05	Custom Two-Step Admin Transfer Should Use OpenZeppelin’s Ownable2Step	Acknowledged
I-01	No Event Emitted When Auto-swap Is Skipped	Acknowledged
I-02	‘setDefaultFees’ Only Affects Future Bonding Curves While Existing Curves Are Silently Unaffected	Acknowledged
G-01	Unbounded Token Arrays Without Pagination Can Lead To DoS Or Large Gas Consumption	Acknowledged

Security Grade



Based on the audit findings and code quality, the overall protocol security grade is **90.00 / 100.00**. This reflects an excellent security posture with almost no room for improvement.

Findings

High

High 01 Auto-swap Reverts On Paused Aerodrome Pool Bricking All Token Transfers

Location

SparkToken.sol

Description

The `SparkToken` contract executes an automatic fee swap inside `_transfer` once the accumulated fee balance exceeds `swapThreshold`. When this condition is met, `_swapAndDistribute` is called, which

internally calls the Aerodrome router function `swapExactTokensForETHSupportingFeeOnTransferTokens`.

Aerodrome V2 pools can be paused by Aerodrome governance. When a pool is paused, swap operations revert at the router level.

Because `_swapAndDistribute` is executed directly inside `_transfer` without a `try/catch` wrapper, any revert occurring inside the router call propagates to `_transfer` and causes the entire transaction to revert.

Since there is no mechanism to disable auto-swap or bypass the swap logic, the token becomes effectively non-transferable for as long as the pool remains paused.

This constitutes a complete liveness failure that can be triggered externally by Aerodrome governance without any malicious action against the Spark contracts.

Proof Of Concept

Scenario demonstrating the failure:

1. Token migrates to an Aerodrome V2 liquidity pool.
2. Trading generates fee tokens that accumulate inside `SparkToken`.
3. The internal balance eventually reaches `swapThreshold`.
4. Aerodrome governance pauses the pool.
5. A user attempts to sell tokens.

Execution path:

```

1  _transfer()
2    └─ condition: balanceOf(address(this)) ≥ swapThreshold
3        └─ _swapAndDistribute()
4            └─ router.swapExactTokensForETHSupportingFeeOnTransferTokens( ... )
5                -> revert (pool paused)

```

Because no `try/catch` is used:

```

1  _swapAndDistribute() revert
2      ↓
3  _transfer() revert
4      ↓
5  transaction fails

```

Impact:

- No user can sell tokens

- No transfers involving non-excluded accounts succeed
- The protocol team cannot bypass the failure

The token remains frozen until the Aerodrome pool is unpaused.

Recommendation

Wrap the swap call inside `_transfer` with a `try/catch` block so that swap failures do not propagate and revert transfers.

Example mitigation:

```
1 if (migrated && !_inSwap && from != v2Pair && balanceOf(address(this)) ≥ swapThreshold)
    {
2     try this._swapAndDistribute() {} catch {}
3 }
```

Additionally, implement an admin function such as `forceSwap(uint256 minOut)` that allows manual fee swaps when conditions are safe.

Status

Fixed

Medium

Mid 01 Ghost `_rOwned` Balance Drift On Excluded Addresses

Location

SparkToken.sol

Description

In `SparkToken`, addresses excluded from reflections maintain two parallel balance trackers: `_tOwned` (the canonical token balance) and `_rOwned` (the reflection-space balance). For excluded addresses, `balanceOf` returns `_tOwned` directly, while `_rOwned` is used solely in `_getCurrentSupply` to correctly account for excluded holdings when computing the global reflection rate.

The bug arises because the reflection rate (`_getRate`) is monotonically decreasing — every taxed trade reduces `_rTotal` while `_tTotal` stays unchanged, pushing the rate down over time. When

an excluded address receives tokens, `_rOwned` is credited at the current (higher) rate. When those same tokens are later transferred out, `_rOwned` is debited at a lower rate. Since `_tOwned` and `_rOwned` are decremented independently, `_tOwned` can reach zero while a phantom residual in `_rOwned` remains.

This ghost balance is never cleared. It permanently corrupts `_getCurrentSupply`, which subtracts each excluded address's `_rOwned` from the global `rSupply`. An artificially deflated `rSupply` with a correct `tSupply` produces an understated rate, which causes `_tokenFromReflection` to return inflated values for all non-excluded holders.

The affected excluded addresses are `bondingCurve`, `factory`, `v2Pair`, `poolFees`, `foundation`, and `address (this)` — several of which process very high token volumes, meaning the drift accumulates rapidly in normal operation.

Recommendation

Resync `_rOwned` to the current rate whenever `_tOwned` is updated for excluded addresses. This must be applied in both `_transferNoFee` and `_transferWithFee`.

`_transferNoFee:`

```
1 function _transferNoFee(address from, address to, uint256 tAmount) private {
2     uint256 rate = _getRate();
3
4     if (isExcludedFromReflections[from]) {
5         _tOwned[from] -= tAmount;
6         _rOwned[from] = _tOwned[from] * rate; // resync to eliminate drift
7     } else {
8         _rOwned[from] -= tAmount * rate;
9     }
10
11     if (isExcludedFromReflections[to]) {
12         _tOwned[to] += tAmount;
13         _rOwned[to] = _tOwned[to] * rate; // resync to eliminate drift
14     } else {
15         _rOwned[to] += tAmount * rate;
16     }
17
18     emit Transfer(from, to, tAmount);
19 }
```

`_transferWithFee:`

```
1 // Debit sender
2 if (isExcludedFromReflections[from]) {
3     _tOwned[from] -= tAmount;
4     _rOwned[from] = _tOwned[from] * rate; // resync
```

```
5 } else {
6     _rOwned[from] -= rAmount;
7 }
8
9 // Credit receiver
10 if (isExcludedFromReflections[to]) {
11     _tOwned[to] += tTransfer;
12     _rOwned[to] = _tOwned[to] * rate; // resync
13 } else {
14     _rOwned[to] += rTransfer;
15 }
```

Zeroing `_rOwned` when `_tOwned` hits zero is not recommended as an alternative, because if an address were ever un-excluded its entire balance would be derived from `_rOwned` — zeroing it would cause permanent, unrecoverable balance loss. The resync approach keeps `_rOwned` semantically valid at all times regardless of future changes to exclusion status.

Status

Acknowledged

Low

Low 01 Plain Text Private Key In Deployment Scripts

Location

DeployMainnet.s.sol

DeployTestnet.s.sol

Description

The deployment scripts directly retrieves the deployer's private key from environment variables using `vm.envUint("DEPLOYER_PRIVATE_KEY")`. This practice exposes sensitive information to multiple security risks:

1. Shell History: Private keys stored in environment variables can be logged in shell command history, making them accessible through `.bash_history`, `.zsh_history`, or similar files.
2. Process Exposure: Environment variables are visible in process listings and system monitoring tools, potentially exposing the key to other users on shared systems.

3. **.env File Leaks:** When using `.env` files to populate environment variables, these files often contain plain-text private keys and can be accidentally committed to version control or exposed through misconfigured backups.
4. **Log Files and Error Messages:** Environment variables can leak into application logs, error traces, or CI/CD pipeline outputs if exceptions occur during script execution.
5. **Memory Exposure:** The private key remains in memory during script execution and could be captured through memory dumps or debugging tools.

This is not a functional vulnerability in the contract logic itself, but rather an operational security issue in the deployment process that could lead to complete loss of funds and control over deployed contracts.

Recommendation

Implement more secure private key management:

1. **Foundry Keystore (Recommended for Local Development):** Use Foundry's built-in key-store with encrypted storage.
2. **Interactive Prompt:** Have the deployer enter the key at runtime rather than from environment variables, keeping it out of shell history and process listings.
3. **Hardware Wallets/Ledger Integration:** Use hardware wallet integrations with Foundry for production deployments.
4. **Secrets Management Service:** For automated deployments, use AWS Secrets Manager, HashiCorp Vault, or similar services with proper access controls and audit logging.

Status

Acknowledged

Low 02 setMcapParams Allows Near-zero Migration Gap Enabling Instant Migration

Location

SparkFactory.sol

Description

The `setMcapParams` function only enforces that:

```
1 migrationMcapInEth > initialMcapInEth
```

This allows extremely small values such as:

```
1 initialMcap = 1 wei
2 migrationMcap = 2 wei
```

With such parameters, the bonding curve's virtual reserves become highly distorted, allowing the migration threshold to be reached with an extremely small buy.

This may trigger liquidity migration before meaningful token distribution or price discovery.

Recommendation

Enforce stronger parameter validation, such as:

- Minimum ratio requirement
- Minimum absolute value requirement

Example:

```
1 require(_migrationMcapInEth ≥ _initialMcapInEth * 5);
2 require(_initialMcapInEth ≥ 0.01 ether);
```

Status

Acknowledged

Low 03 swapThreshold Fixed At Deployment And Becomes Misaligned With Circulating Supply

Location

SparkToken.sol

Description

The `swapThreshold` value is initialized during deployment as:

```
1 TOTAL_SUPPLY / 10_000
```

However, the token supply can decrease over time due to burns during migration and reflection mechanics.

Because `swapThreshold` remains constant while total supply decreases, the threshold gradually represents a larger portion of circulating supply. This causes each swap to execute progressively larger trades relative to the pool size.

Large swaps increase price impact and may reduce the value recovered from fees.

Recommendation

Allow the swap threshold to be updated by governance with proper bounds, or compute the threshold dynamically using the current `totalSupply`.

Status

Acknowledged

Low 04 `confirmDeposit` Lacks `nonReentrant` Protection

Location

KickoffFoundation.sol

Description

The `confirmDeposit` function calls an external contract (`voteSalePool`) but does not include the `nonReentrant` modifier, while similar functions in the contract are protected.

This inconsistency introduces a potential reentrancy risk if the external contract ever invokes callbacks.

Recommendation

Apply the `nonReentrant` modifier to `confirmDeposit` for consistency and defense-in-depth.

Status

Fixed

Low 05 Custom Two-Step Admin Transfer Should Use OpenZeppelin's Ownable2Step

Location

SparkFactory.sol

KickoffFoundation.sol

SparkLPLocker.sol

Description

All three contracts implement a bespoke two-step admin transfer pattern — `transferAdmin` sets a `pendingAdmin`, and `acceptAdmin` completes the handover. This is functionally equivalent to OpenZeppelin's `Ownable2Step`, but the custom implementation has a few gaps compared to the battle-tested standard:

- There is no way to cancel a pending transfer. If `transferAdmin` is called with an incorrect address, the only recovery is to call `transferAdmin` again with the correct one — this is non-obvious and undocumented.
- The pattern is reimplemented independently across three contracts with no shared base, meaning any future bug fix or improvement must be applied in three places.
- The custom implementation is less immediately recognisable to external auditors and integrators who are familiar with the OpenZeppelin standard.

The one legitimate reason to keep the pattern custom is that `SparkFactory.acceptAdmin` has a side effect — it calls `SparkLPLocker.syncAdmin(msg.sender)` to keep the two contracts in sync. This cannot be done with `Ownable2Step` out of the box.

Recommendation

Inherit from OpenZeppelin's `Ownable2Step` across all three contracts. For `SparkFactory`, override `_transferOwnership` to include the `syncAdmin` side effect:

```
1 function _transferOwnership(address newOwner) internal override {  
2     super._transferOwnership(newOwner);  
3     SparkLPLocker(lpLocker).syncAdmin(newOwner);  
4 }
```

This retains the sync behaviour while gaining the audited, standardised, and cancellable ownership transfer pattern. Also consider whether `renounceOwnership` should be disabled given the protocol's reliance on an active admin.

Status

Acknowledged

Informational

Info 01 No Event Emitted When Auto-swap Is Skipped

Location

SparkToken.sol

Description

The `_swapAndDistribute` function exits silently under several conditions without emitting an event.

This prevents off-chain monitoring systems from detecting skipped swaps or accumulated fee tokens.

Recommendation

Emit a `SwapSkipped` event for each early-exit path.

Status

Acknowledged

Info 02 `setDefaultFees` Only Affects Future Bonding Curves While Existing Curves Are Silently Unaffected

Location

SparkFactory.sol

Description

`setDefaultFees` updates `defaultReflectionFeeBps`, `defaultCreatorFeeBps`, and `defaultFoundationFeeBps` on the factory. However, these values are only read at token creation time — they are passed as constructor arguments into each `SparkBondingCurve` and `SparkToken` deployment and stored as `immutable` values on those contracts. Any fee change made via `setDefaultFees` has zero effect on any bonding curve or token that already exists.

This creates a silent divergence: the factory's displayed fee parameters no longer represent the fees that live tokens are actually charging. An operator or user reading `defaultCreatorFeeBps` from the factory after a fee update would incorrectly believe all tokens are operating under the new fee schedule.

The risk compounds if the admin reduces fees intending to make the protocol more competitive — existing tokens continue charging the old higher fees, potentially driving users away from live tokens while new ones launch at the lower rate. The inverse is also true: a fee increase does not apply to existing tokens, so there is no way to uniformly raise fees across the protocol.

There is no event, no `NatSpec`, and no on-chain indication that existing deployments are unaffected by this change.

Recommendation

Add explicit `NatSpec` documentation to `setDefaultFees` making the scope of the change clear:

```
1 /// @notice Updates default fees for future token deployments only.
2 /// @dev Has NO effect on bonding curves or tokens already deployed.
3 ///     Each deployed SparkToken and SparkBondingCurve stores fees
4 ///     as immutables set at construction time.
5 function setDefaultFees(...) external onlyAdmin { ... }
```

If uniform fee updates across all live tokens are a protocol requirement, the architecture needs to change — fees on `SparkToken` and `SparkBondingCurve` would need to be mutable and governor-controlled rather than immutable. Consider also emitting the old fee values alongside the new ones in `DefaultFeesUpdated` so the change is fully auditable on-chain.

Status

Acknowledged

Gas

Gas 01 Unbounded Token Arrays Without Pagination Can Lead To DoS Or Large Gas Consumption

Location

SparkLPLocker.sol

SparkFactory.sol

Description

The arrays `allTokens` and `allLockedTokens` grow indefinitely as new tokens are launched. If future functions iterate over these arrays on-chain, this may cause gas-related denial-of-service issues. Off-chain clients may also encounter performance degradation when fetching large arrays.

Recommendation

Provide paginated getter functions that allow partial retrieval of array data. This saves gas and mitigates the possibility of a denial-of-service issues.

Status

Acknowledged